

General AI Fluency · Impact Project Capstone (FL-CAP)

Author: Harshit Kudhial (@harshitttt077) | Track: General AI Fluency | System: Living Portfolio Engine | Date: September 2026

1. System Purpose: Turning One-Off Projects into a Compounding Platform

A portfolio that never receives a second project degrades from an active proof engine into a static academic artifact. The difference between a transient class project and a permanent career platform is an operational system that makes publishing the next case study friction-free. This capstone establishes the mechanical habit, directory architecture, and preserved AI context required to ship subsequent case studies in under 15 minutes.

2. Concrete 'How to Add the Next Case' SOP (3-Beat Narrative Shape)

Dimension	Portfolio System Specification
File Location	/content/case-studies/02-distributed-crawler.mdx inside Next.js App Router repository. Dynamic router loads via app/(portfolio)/work/[slug]/page.tsx.
Beat 1: The Problem <i>(Stakes & Failure Mode)</i>	Document operational constraints, what broke under real traffic (rate limits, lock contention, memory leaks), and the financial/latency cost of naive approaches.
Beat 2: What You Did <i>(Architecture & Trade-offs)</i>	Specific tech stack, explicit design trade-offs (e.g. Go channels vs mutexes, Redis Sorted Sets vs Kafka), anti-patterns avoided, and one high-signal architecture snippet.
Beat 3: What Came of It <i>(Empirical Proof)</i>	Structured quantitative before/after benchmark table: throughput, p99 latency, error rates, resource utilization, and key engineering takeaways.

3. Named Next Real Piece of Work: Case Study 2 Specification

Project Element	Engineering Implementation Details
Project Title	High-Throughput Distributed Web Crawler & Search Ingestion Engine
Repository & Stack	github.com/harshitttt077/distributed-crawler-engine Go (Golang), Redis (Sorted Sets), DuckDB, gRPC, Docker.
Alignment with Proof	Fulfills core proof statement: 'I build high-throughput concurrent systems and ML pipelines under real-world constraints.' Moves from decision logic (ML refresh) to underlying data ingestion.
Beat 1 (Problem)	Fetching 250k+ web pages daily across thousands of client domains triggered severe CDN 429 rate-limits / IP blocks, or created a 36-hour crawl queue backlog when throttled naively.
Beat 2 (What I Did)	Engineered a decentralized concurrent crawler in Go using per-domain token buckets, bounded goroutine worker pools, Redis Sorted Sets for priority scheduling, and partitioned Parquet/DuckDB storage.
Beat 3 (Outcome)	Sustained 4,200 pages/min across 1,200 domains with <0.02% 429 error rate . Queue clearance dropped from 36 hours to 1h 45m with zero worker memory leaks over 72h continuous operation.

4. Concrete Reminder Infrastructure & Evidence

To ensure the publishing habit is executed systematically rather than left as a vague hope, an automated multi-stage calendar schedule has been established and exported as an RFC 5545 standard file (outputs/calendar_reminder_case_study_2.ics).

Field	Scheduled Cadence & Payload
Event Title	[Portfolio Maintenance] Ship Case Study 2: Distributed Crawler
Target Execution Date	Sunday, October 4, 2026 at 10:00 AM – 11:30 AM IST (3 weeks post-internship wrap).
Recurrence Rule	Repeats Monthly on the first Sunday (RRULE:FREQ=MONTHLY;BYDAY=1SU).
Multi-Stage Alerts	• T-48 Hours: Draft review prompt in Claude Project using raw benchmark logs. • T-2 Hours: Final MDX render verification, Next.js build check, git push.
Verification File	Standard iCalendar export: outputs/calendar_reminder_case_study_2.ics.

5. Preserved Claude Project Build Context (Cheap Updates)

The Claude Project configured in Week 1 preserves Harshit's identity, technical proof, and MDX component syntax. When the October 4 reminder fires, no context re-building is required. Harshit pastes raw benchmarks into the project with this prompt:

[REUSABLE CASE STUDY GENERATOR PROMPT]

Project: Harshit Kudhial - Engineering Portfolio

Task: Generate Case Study 2 MDX from raw crawler project notes.

- Project: High-Throughput Distributed Web Crawler & Search Ingestion Engine
- Stack: Go, Redis Sorted Sets, DuckDB, Docker, gRPC
- Raw Benchmarks: Old backlog: 36h, 14% 429 ban rate, OOM crash at 4h. New: Token-bucket per host, bounded goroutines, 4,200 pages/min, <0.02% 429 rate, 1h 45m queue clear, 72h memory stability.
- Output: Format into 3-Beat MDX (/content/case-studies/02-distributed-crawler.mdx) with frontmatter and quantitative metric cards.

6. Pass / Revise Assessment Rubric

Evaluation Requirement	Fulfillment Evidence	Result
1. Concrete 'how to add' note	Explicit MDX file path, Next.js dynamic routing, and section-by-section 3-beat narrative guide.	PASS
2. Named specific next piece	Distributed Web Crawler & Ingestion Engine (Go/Redis/DuckDB) with drafted 3 beats and metrics.	PASS
3. Evidence of reminder set	RFC 5545 .ics calendar file generated with October 4, 2026 trigger, monthly recurrence & 2 alerts.	PASS
4. Build context preserved	Preserved Claude Project knowledge kit + single-turn generator prompt for <15 min updates.	PASS